Lab 5：缺页异常与 Fork 机制

3230104220 徐晟洋
2025.12.15

分工说明：队友周皓瑜负责Task1-2，我负责Task3-4

---

1 Part 1 (VMA)

1.1 设计目的

我们想实现按需加载，所以设计VMA用于判断内存意图，通过页表只能看到物理内存访问是否有效。
当我们需要访问的时候，再根据VMA把内容从磁盘中加载进来（在我们的代码中是虚拟磁盘）

1.2 具体实现：Task1

首先需对与 task->pgd 相关的代码进行调整，将其修改为 task->mm.pgd 的形式，以此适配页表被整合至内存结构体这一变更。

1.2.1 补全 find_vma

如下图所示，我们借鉴 free_vma 的实现思路，采用侵入式链表的方式遍历全部虚拟内存区域（VMA），同时校验目标地址是否处于该 VMA 的地址区间内，一旦匹配到对应的 VMA 便立即将其返回。

```c
struct vm_area_struct *find_vma(struct mm_struct *mm, uint64_t addr)
{
    /* Lab5 Task1 */
    // 仿照free_vma枚举每个VMA
    struct vm_area_struct *vma;
    struct list_head *pos, *n;

    list_for_each_safe(pos, n, &mm->mmap)
    {
        vma = list_entry(pos, struct vm_area_struct, list);
        if(vma->vm_start <= addr && addr <= vma->vm_end)
            return vma;
    }

    return NULL;
}
```

1.2.2 补全 do_mmap

该函数的核心功能为：为 VMA 申请一块内存空间，初始化其侵入式链表节点，将该 VMA 节点加入进程的 VMA 链表中，同时根据传入的参数（如地址、长度、标志位等）完成 VMA 各成员变量的赋值操作，最终返回 VMA 的起始地址。

```
/* Lab5 Task1 */
// 初始化本VMA的侵入式链表并将其链接至VMA链表中
INIT_LIST_HEAD((&vma->list));
list_add(&vma->list,&mm->mmap);

// 计算其余字段
vma->vm_start = addr;
vma->vm_end = addr + len;
vma->vm_flags = flags;
vma->vm_file = file;
vma->vm_filesz = vm_filesz;
vma->vm_pgoff = vm_pgoff;
```

1.2.3 修改 proc.c 文件，完成 VMA 生命周期维护

1. **VMA 链表初始化**：在 task_init 函数中添加以下代码，实现对进程 VMA 链表的初始化：

   ```
   INIT_LIST_HEAD(&init_task->mm.mmap);
   ```

2. **VMA 链表拷贝**：在 copy_process 函数中添加以下代码，完成子进程对父进程 VMA 链表的拷贝：

   ```
   INIT_LIST_HEAD(&new_task->mm.mmap);
   copy_vma(&new_task->mm, &old->mm);
   ```

3. **VMA 链表释放**：在 release_task 函数中添加以下代码，实现进程退出时 VMA 链表的资源释放：

   ```
   free_vma(&task->mm);
   ```

2 Part 2 (Demand Paging)

2.1 设计目的

如何在程序毫不知情的情况下，只在它真正用到某页内存的那一瞬间，才给它分配物理内存。

所以说要分两部分：

1. 先设套 (修改 load_elf_binary)：当有内存需求的时候，不再直接分配物理内存，而是在VMA上记录下来。
2. 之后解套 (实现 do_page_fault)：由于你没有分配实际物理页，所以如果访问肯定会报错。所以按照以下流程来鉴别：
   (a) 查看报错原因：是否是page fault
   (b) 读取stval中的虚拟地址，并去VMA中比对，看看访问是否合法
   (c) 如果合法，就还它空间，alloc_page()，并create_mapping，并刷新TLB

2.2 具体实现：Task2

2.2.1 修改 load_elf_binary 函数

```c
// 遍历每个phdr
for(uint64_t i = 0; i < ehdr->e_phnum; i++)
{
    // 取出当前phdr
    Elf64_Phdr *nw_phdr = &phdr[i];

    // 若需要加载，为该段创建VMA加入VMA链表
    if(nw_phdr->p_type == PT_LOAD)
        do_mmap(&task->mm, nw_phdr->p_vaddr, nw_phdr->p_memsz, nw_phdr->p_offset,
        nw_phdr->p_filesz, flags_phdr_to_vma(nw_phdr->p_flags), file);
}
/* Lab5 Task2 */
// user_stack
do_mmap(/* Lab5 Task2*/ &task->mm, VM_USER_STACK_TOP - PGSIZE,
    PGSIZE, 0, 0, VM_READ | VM_WRITE, NULL);

start_thread(/* Lab4 Task3 */ task_pt_regs(task), ehdr->e_entry, VM_USER_STACK_TOP);
```

如上图所示，该函数的核心逻辑为：

遍历 ELF 文件的所有 Program Header，当检测到段类型为需要加载的类型（如 PT_LOAD）时，调用 do_mmap 函数创建对应的 VMA 映射并加载段内容；

此外，额外增加用户栈的 VMA 映射，将 vm_file 参数设为 NULL 以标识该 VMA 为匿名区域映射，并将文件相关的参数（如 vm_pgoff、vm_filesz）置为 0。

2.2.2 补全 do_page_fault 函数

```c
        // 将多余的部分置零
        memset(new_page, 0, offset);
        memset(new_page + offset + size, 0, PGSIZE - offset - size);

        LOG_DEBUG("copy: dest=%p src=%p siz=%ld\n", new_page + offset,      You, 6天前 • Lab5 finish to Task2,final
            vma->vm_file + vma->vm_pgoff + offset, size);

        create_mapping(curr->mm.pgd, (void *)vma->vm_start + offset + pg_num * PGSIZE, (void *)VA2PA(new_page),
            size, flags_vma_to_pte(vma->vm_flags) | PTE_U);

    }
    else
    {
        memset(new_page, 0, PGSIZE);

        // 创建页表映射并设置内存屏障（我说当时还仔细学习了sfence的种种参数版本，结果现在就这样用）
        create_mapping(curr->mm.pgd, (void *)bad_addr, (void *)VA2PA(new_page),
            PGSIZE, flags_vma_to_pte(vma->vm_flags) | PTE_U);
    }

    asm volatile("sfence.vma");
```

如上图所示，该版本函数实现了缺页异常的基础处理逻辑：通过 bad_addr 找到对应的 VMA，在校验权限无误但未找到对应页表项时，申请新的物理页并拷贝内容，最后建立页表映射。

以下部分为我的主要工作

3 Part 3 (Fork机制)

3.1 设计目的

fork的目的是创建一个和当前一模一样的进程

一方面要复制相关资源，而且子进程被唤醒后，也要停在fork的地方。而且要实现“一次调用，两次返回”

3.2 具体实现：Task3

```c
long sys_clone(const struct pt_regs *regs)
{
    struct task_struct *new_task = copy_process(current);

    /* Lab5 Task3 */
    new_task->thread.ra = (uint64_t)ret_from_trap;
    new_task->thread.sp = (uint64_t)task_pt_regs(new_task);
    new_task->wait_comp = NULL;

    struct pt_regs *child_regs = task_pt_regs(new_task);
    *child_regs = *regs;
    child_regs->a0 = 0;
    child_regs->sepc = regs->sepc + 4;

    LOG_INFO("fork new process [pid = %" PRIu64 "]\n", new_task->pid);

    return new_task->pid;
}
```

调用流程如下

1. 复用 copy_process：在 sys_clone 中调用 copy_process。，用于处理task_struct 分配、PID 分配、页表复制（copy_pgd）和 VMA 复制。
2. 构造子进程的内核栈：
   (a) 父进程进入 sys_clone 时，其寄存器状态保存在内核栈顶的 pt_regs 结构中。
   (b) 我们需要将父进程的 pt_regs 完全拷贝到子进程的内核栈顶。这是为了让子进程醒来时，拥有和父进程一样的用户态上下文（PC, SP, 通用寄存器等）。
3. 伪造上下文切换现场：
   (a) 设置子进程的 thread.ra 指向 ret_from_trap（汇编函数）。
   (b) 设置子进程的 thread.sp 指向子进程内核栈保存 pt_regs 的位置。
   (c) 效果：当调度器切换到子进程时，它会跳转到 ret_from_trap，从栈中恢复我们刚刚拷贝的 pt_regs，从而“假装”从系统调用返回到用户态。
4. 区分返回值：
   (a) 父进程：sys_clone 函数正常返回子进程的 PID。
   (b) 子进程：修改子进程 pt_regs 中的 a0 寄存器为 0。这样子进程回到用户态时，看到的返回值就是 0。
5. 修正 sepc：确保子进程的 sepc 指向 ecall 的下一条指令（ regs->sepc + 4），防止子进程醒来后死循环执行 ecall。

4 Part4（CoW）

4.1 设计目的

Copy on write, 即fork时不再立即深拷贝所有物理内存。因为子进程往往紧接着调用 exec替换内存空间，只有当父子进程真正尝试修改内存时，才分配新的物理页并复制数据。

4.2 具体实现：Task4

4.2.1 修改页表复制 (vm.c)

```c
            /* Lab5 Task4 */
            uint64_t flags = old_l0_pte & 0x3FF;
            if (flags & PTE_W) {
                flags &= ~PTE_W;
                old_l0_pgd[k] = (old_l0_pte & ~0x3FF) | flags;
            }

            l0_pgd[k] = (old_l0_pte & ~0x3FF) | flags;

            void *page_va = (void *)PA2VA(PTE2PA(old_l0_pte));
            get_page(page_va);
        }
    }
}

asm volatile("sfence.vma");
```

此处修改copy过程，原本实现为深拷贝，现在改为浅拷贝，所以我们不再分配page，而是把pid记下来，并且将权限设置为可读（进而引出后文缺页异常）

需要注意的是，此处一定要刷新TLB，简单来说，刷新了页表，Cache肯定也要更新。

4.2.2 修改缺页异常处（trap.c）

```c
if (ptep && (*ptep & PTE_V)) {
    if (scause == SCAUSE_SPF) { // CoW: PTE 存在且有效，写缺页错误
        // 没必要检查 W 了，肯定是没有的
        LOG_DEBUG("CoW at 0x%lx\n", bad_addr);
        /* Lab5 Task4 */
        uint64_t pte = *ptep;
        void *page_va = (void *)PA2VA(PTE2PA(pte));
        if (refcnt_page(page_va) > 1) {
            void *new_page_va = alloc_page();
            memcpy(new_page_va, page_va, PGSIZE);
            uint64_t new_page_pa = VA2PA((uint64_t)new_page_va);
            *ptep = ((new_page_pa >> 12) << 10) | (pte & 0x3FF) | PTE_W;
            put_page(page_va);
        } else {
            *ptep |= PTE_W;
        }
        asm volatile("sfence.vma");
        return;
```

当任一进程尝试写入这些被标记为“只读”的共享页时，CPU 会触发 Store Page Fault

接下来进行判定，检查：

1. 出错原因是否为写异常 (SCAUSE_SPF)。
2. 该地址的 PTE 是否有效 (PTE_V) 但不可写。

之后进行分类讨论：

如果引用计数 > 1：说明有多个进程共享，此时需要：

1. 分配一个新的物理页。
2. 将旧页面的内容拷贝到新页面。
3. 修改当前进程的 PTE 指向新页面，并开启写权限 (PTE_W)。
4. 旧物理页引用计数减 1。

如果引用计数 == 1：说明只剩当前进程在使用（其他进程可能已经 COW 分离了或退出了）。那么直接将当前 PTE 的写权限 (PTE_W) 恢复即可，无需拷贝。